Chương 2. Các mô hình dữ liệu

Mô hình dữ liệu quan hệ và ánh xạ ER $\rightarrow$ quan hệ

Đỗ Thanh Hà

Khoa Trí tuệ nhân tạo
Học viện Công nghệ Bưu chính Viễn thông

Ngày 18 tháng 9 năm 2026

Nội dung

- Giới thiệu
- Quá trình thiết kế và cài đặt CSDL
- Mô hình thực thể — liên kết (ER)
- Một số vấn đề cần quan tâm khi thiết kế lược đồ E-R
- **Mô hình dữ liệu quan hệ**
  - Định nghĩa hình thức, các khái niệm, đặc tính của quan hệ
  - Khóa và các ràng buộc toàn vẹn
- **Ánh xạ mô hình ER sang mô hình quan hệ** — thuật toán 7 bước

Mục tiêu

Sau bài học, sinh viên có thể: (1) phát biểu định nghĩa hình thức của quan hệ và giải thích các đặc tính của nó; (2) xác định khóa chính, khóa ngoại và ba ràng buộc toàn vẹn; (3) áp dụng đúng thuật toán 7 bước để chuyển một lược đồ ER bất kỳ thành lược đồ quan hệ kèm ràng buộc.

Mô hình dữ liệu quan hệ

- Mô hình quan hệ do **E. F. Codd** — một nhà toán học làm việc tại IBM — đề xuất năm **1970** trong bài báo *"A Relational Model of Data for Large Shared Data Banks"*.
- Nền tảng toán học: **lý thuyết tập hợp** và **logic vị từ bậc nhất** (*first-order predicate logic*).
- Đây là mô hình *hướng bản ghi*, thuộc **mức logic** — khác với mô hình ER ở mức khái niệm đã học ở bài trước.

Vì sao nền tảng toán học lại quan trọng?

Chính vì quan hệ là một *tập hợp* nên ta suy ra được toàn bộ các đặc tính (không trùng bộ, không có thứ tự). Và chính vì có logic vị từ nên mới có *đại số quan hệ* và *phép tính quan hệ* — cơ sở lý thuyết để hệ quản trị CSDL **tối ưu hóa** câu truy vấn SQL một cách tự động.

Định nghĩa hình thức

- Cho các **miền giá trị** $D_1, D_2, \ldots, D_n$ (mỗi miền là một tập giá trị nguyên tố, ví dụ `CHAR(8)`, `DATE`).
- **Lược đồ quan hệ:** $R(A_1{:}D_1,\ A_2{:}D_2,\ \ldots,\ A_n{:}D_n)$ — tên quan hệ, danh sách thuộc tính kèm miền giá trị và các ràng buộc.
- **Một quan hệ** (thể hiện) r trên lược đồ R là một tập hữu hạn:

$$r \subseteq D_1 \times D_2 \times \cdots \times D_n$$

  tức mỗi phần tử $t \in r$ là một **bộ** $t = \langle v_1, \ldots, v_n \rangle$ với $v_i \in D_i$.
- **Bậc** (degree) $= n$ (số thuộc tính); **lực lượng** (cardinality) $= |r|$ (số bộ).

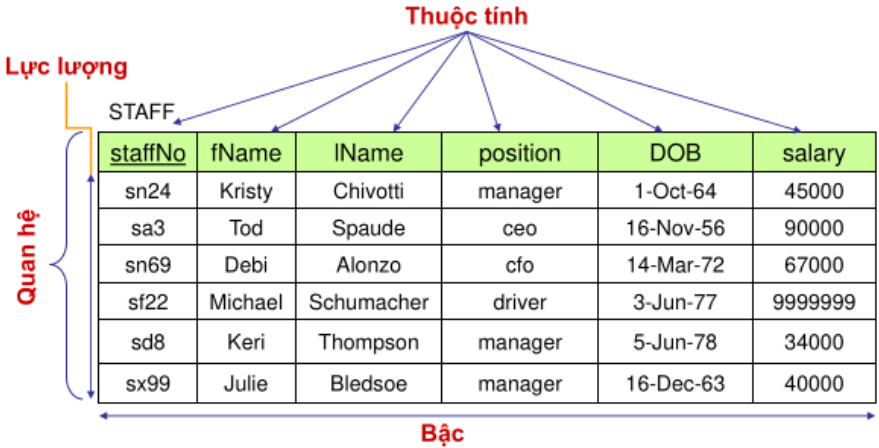
Hệ quả: Toàn bộ "đặc tính của quan hệ" đến từ đây

r là một **tập hợp**, nên: (a) không thể có hai bộ giống hệt nhau; (b) Không có khái niệm "bộ thứ nhất, bộ thứ hai". Và nếu ta coi mỗi bộ là một *ánh xạ* từ *tên thuộc tính* sang giá trị, thì thứ tự cột cũng vô nghĩa.

Các khái niệm cơ bản

- **Quan hệ**: một *tập các bộ* cùng cấu trúc, lưu thông tin về các đối tượng được mô hình hóa. Ta *biểu diễn* nó dưới dạng bảng có hàng và cột cho dễ nhìn.
- **Thuộc tính**: các cột được đặt tên trong quan hệ. Mỗi thuộc tính mô tả một đặc tính của thực thể hoặc liên kết được mô hình hóa.
- **Miền giá trị**: tập các giá trị hợp lệ; mỗi thuộc tính được xác định trên đúng một miền. Nhiều thuộc tính có thể dùng chung một miền (ví dụ `NgaySinh` và `NgayVaoLam` cùng miền `date`).
- **Bộ**: một hàng của quan hệ.
- **Bậc**: số thuộc tính. **Lực lượng**: số bộ.
- **CSDL quan hệ**: tập các quan hệ có tên phân biệt, mỗi quan hệ đã *chuẩn hóa* (ở đây nghĩa là đạt **dạng chuẩn 1** - mọi giá trị đều nguyên tố), cùng với các ràng buộc giữa chúng.

Ví dụ về quan hệ



staffNo	fName	lName	position	DOB	salary
sn24	Kristy	Chivotti	manager	1-Oct-64	45000
sa3	Tod	Spaude	ceo	16-Nov-56	90000
sn69	Debi	Alonzo	cfo	14-Mar-72	67000
sf22	Michael	Schumacher	driver	3-Jun-77	9999999
sd8	Keri	Thompson	manager	5-Jun-78	34000
sx99	Julie	Bledsoe	manager	16-Dec-63	40000

bậc đo theo chiều ngang (số cột), *lực lượng* đo theo chiều dọc (số hàng); thuộc tính khóa được gạch chân.

Miền giá trị cho ví dụ trên

Thuộc tính	Tên miền	Ý nghĩa	Định nghĩa miền
staffNo	staffnumbers	Tập của tất cả các số hiệu có thể có của nhân viên	Kiểu ký tự: kích cỡ 4, phải bắt đầu bằng chữ s.
fName, lName	name	Tập tất cả các tên có thể có của một người	Kiểu ký tự: kích cỡ 20
DOB	date	Ngày sinh của một người	Kiểu ngày tháng: trong khoảng từ 1-Jan-20, Khuôn dạng: dd-mm-yy
salary	salaries	Các giá trị có thể có của lương nhân viên	Dạng tiền tệ: 7 ký tự, trong khoảng 10,000-9,999,999
position	alljobs	Tập tất cả các vị trí có thể có của một nhân viên trong công ty	Chọn một trong các tập: : {ceo, cfo, coo, manager, asst. manager, driver, secretary}

Chú ý: miền giá trị không chỉ là *kiểu dữ liệu*. Nó còn gồm khuôn dạng và tập giá trị hợp lệ - và chính phần này về sau được cài đặt bằng `CHECK` hoặc kiểu liệt kê trong SQL.

Các thuật ngữ tương đương

Thuật ngữ hình thức	Cách gọi thông dụng	Cách gọi kiểu tệp
Quan hệ (relation)	Bảng (table)	Tệp (file)
Bộ (tuple)	Hàng (row)	Bản ghi (record)
Thuộc tính (attribute)	Cột (column)	Trường (field)

Thuật ngữ chuẩn	Lựa chọn 1	Lựa chọn 2
Quan hệ	Bảng	Tệp
Bộ	Hàng	Bản ghi
Thuộc tính	Cột	Trường

Các đặc tính của một quan hệ

1. Quan hệ có tên phân biệt với mọi quan hệ khác trong cùng lược đồ CSDL.
2. Mỗi thuộc tính có tên riêng, phân biệt trong phạm vi một quan hệ.
3. Mỗi thuộc tính xác định trên một miền giá trị.
4. Mỗi ô chứa **một giá trị nguyên tố** (không chia nhỏ, không phải danh sách).
5. Các bộ **phân biệt nhau** — không có hai bộ giống hệt nhau.
6. Thứ tự các thuộc tính không quan trọng.
7. Thứ tự các bộ không quan trọng (về mặt lý thuyết).

Nguồn gốc của từng đặc tính

Các đặc tính (5), (6), (7) *không phải quy ước* mà là hệ quả toán học của việc quan hệ là một tập hợp các ánh xạ tên-thuộc-tính $\to$ giá trị. Đặc tính (4) chính là **dạng chuẩn 1** (1NF).

$\Longrightarrow$ Trong thực tế, thứ tự vật lý của các bộ trên đĩa *có* ảnh hưởng tới hiệu quả truy nhập — nhưng đó là chuyện của mức trong, không phải của mô hình.

Quan hệ (lý thuyết) và bảng SQL không giống nhau

	Quan hệ (mô hình)	**Bảng trong SQL**
Bộ trùng lặp	Không thể tồn tại	Được phép, trừ khi có `PRIMARY KEY`/`UNIQUE`
Thứ tự cột	Vô nghĩa	Có nghĩa (`SELECT *`, `INSERT` không nêu cột)
Thứ tự hàng	Vô nghĩa	Không đảm bảo, trừ khi có `ORDER BY`
Giá trị `NULL`	Không có trong mô hình gốc	Có, kéo theo logic **ba giá trị**

Chú ý:

SQL là *xấp xỉ kỹ thuật* của mô hình quan hệ, không phải là hiện thực hóa hoàn hảo. Nhiều lỗi khó hiểu khi làm bài tập xuất phát từ chỗ này: `SELECT` không có `DISTINCT` trả về *đa tập* (multiset) chứ không phải tập hợp; `NULL = NULL` cho kết quả `UNKNOWN` chứ không phải `TRUE`.

Giá trị nguyên tố - hai kiểu vi phạm

Ví dụ: bảng `SinhVien`

Vi phạm kiểu 1: giá trị *phức hợp* (gộp nhiều thành phần)

MaSV	HoTen	NgaySinh	DiaChi
SV01	An	2000-01-01	Số 12, Phố Huế, Hai Bà Trưng, Hà Nội
SV02	Bình	2001-05-12	23 Nguyễn Trãi, Thanh Xuân, Hà Nội

Cách sửa: *tách thành nhiều cột* — `SoNha`, `Duong`, `Quan`, `ThanhPho`. Khi đó mới lọc được "tất cả SV ở Hà Nội".

Vi phạm kiểu 2: giá trị *đa trị* (nhiều giá trị trong một ô)

MaSV	HoTen	SoDienThoai
SV01	An	0912xxxxxx, 0987xxxxxx

Cách sửa: *tách thành bảng riêng* `DienThoaiSV(MaSV, SoDienThoai)` — *không* tạo các cột `SDT1`, `SDT2`, `SDT3`.

Hai kiểu vi phạm, hai cách sửa khác nhau

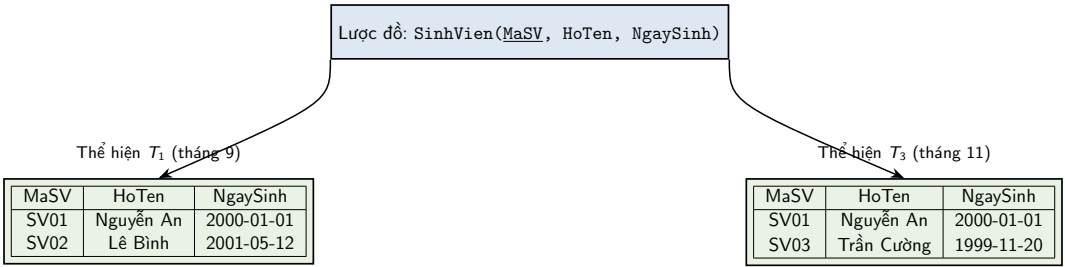
Lược đồ quan hệ và thể hiện quan hệ

- **Lược đồ** = tên quan hệ + tập thuộc tính (kèm miền giá trị) + các ràng buộc. Lược đồ *hầu như không thay đổi* theo thời gian.
- **Thể hiện** = tập các bộ hiện có tại một thời điểm. Thể hiện *thay đổi liên tục*.
- Mọi thao tác *chèn*, *sửa*, *xóa* đều làm thay đổi thể hiện, nhưng không đụng tới lược đồ. Thay đổi lược đồ cần lệnh `ALTER` — thuộc nhóm DDL.
- Một **CSDL hiện thời** (*snapshot database*) phản ánh trạng thái của thế giới thực tại một thời điểm; thế giới thực đổi thì CSDL phải đổi theo để giữ đúng biểu diễn.

Đối chiếu với các khái niệm đã học

Cặp *lược đồ / thể hiện* ở đây tương ứng chính xác với cặp *kiểu thực thể / tập thực thể* trong mô hình ER, và với cặp *intension / extension* nói chung. Mọi ràng buộc đều phát biểu trên *lược đồ* và phải đúng với *mọi* thể hiện hợp lệ.

Lược đồ và thể hiện (minh họa)



Ghi chú: lược đồ cố định; các thể hiện thay đổi theo thời gian. Ràng buộc `MaSV` là khóa chính phải đúng với *cả hai* thể hiện và với mọi thể hiện tương lai.

Sự tương đương của các thể hiện

- **Tương đương** ($\equiv$): hai thể hiện tương đương nếu có *cùng tập thuộc tính* và *cùng tập bộ*.
- **Tính bất biến**: đảo thứ tự hàng hoặc thứ tự cột *không* làm thay đổi thông tin mà quan hệ biểu diễn.

Trường hợp 1: $T_1 \equiv T_2$ (đảo hàng/cột)

T_1

MaSV	HoTen	NS
SV01	Ng. An	2000
SV02	Lê Bình	2001

$\Longleftrightarrow$

T_2

HoTen	NS	MaSV
Lê Bình	2001	SV02
Ng. An	2000	SV01

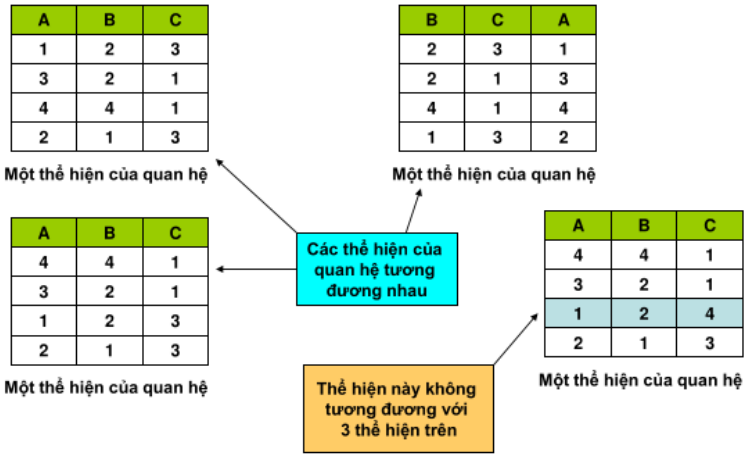
Tập hợp dữ liệu không đổi $\Rightarrow$ *cùng một quan hệ.*

Trường hợp 2: $T_1 \not\equiv T_3$

MaSV	HoTen	NS
SV01	Ng. An	2000
SV02	Lê Bình	2001
SV03	Tr. Cường	1999

Xuất hiện bộ mới $\Rightarrow$ *tập hợp đã thay đổi.*

Các quan hệ tương đương



Bài tập: ba thể hiện đầu tương đương nhau — hãy chỉ ra *chính xác* bộ nào trong thể hiện thứ tư không có mặt ở ba thể hiện kia.

Khóa trong mô hình quan hệ

Các khái niệm đã học ở mô hình ER được giữ nguyên, phát biểu lại trên quan hệ:

- **Siêu khóa** (superkey): tập thuộc tính K sao cho không có hai bộ nào trùng nhau trên K, ở *mọi* thể hiện hợp lệ.
- **Khóa dự bị** (candidate key): siêu khóa *tối thiểu* — không có tập con thực sự nào của nó còn là siêu khóa.
- **Khóa chính** (primary key): một khóa dự bị được chọn; các khóa dự bị còn lại là **khóa thay thế** (alternate key).
- **Khóa ngoại** (foreign key): tập thuộc tính FK trong quan hệ R_2 *tham chiếu* tới khóa chính của quan hệ R_1.

Điều kiện của khóa ngoại

1. Các thuộc tính của FK phải có **miền giá trị tương thích** với khóa chính của R_1 (tên cột thì không nhất thiết giống).
2. Với mỗi bộ $t_2 \in r_2$: hoặc $t_2[FK] = \texttt{NULL}$, hoặc *tồn tại* một bộ $t_1 \in r_1$ với $t_1[PK] = t_2[FK]$.
3. R_1 và R_2 **có thể là cùng một quan hệ** — khi đó ta có *khóa ngoại đệ quy* (ví dụ `MaNguoiGiamSat` trong `NHANVIEN`).

[Lưu ý] Toàn bộ nửa sau của bài giảng dựa vào khóa ngoại.

Ba ràng buộc toàn vẹn của mô hình quan hệ

1. **Ràng buộc miền giá trị** (*domain constraint*): mỗi giá trị của thuộc tính A_i phải thuộc miền D_i và là giá trị nguyên tố.
2. **Toàn vẹn thực thể** (*entity integrity*): không thuộc tính nào thuộc khóa chính được nhận giá trị `NULL`. *Lý do:* khóa chính dùng để định danh — một bộ không định danh được thì vô nghĩa.
3. **Toàn vẹn tham chiếu** (*referential integrity*): mọi giá trị khóa ngoại phải `NULL` hoặc trỏ tới một bộ *đang tồn tại* ở quan hệ được tham chiếu. *Lý do:* không cho phép "tham chiếu treo".

Ngoài ba ràng buộc trên

Còn có **ràng buộc ngữ nghĩa** do nghiệp vụ quy định ("lương không âm", "ngày kết thúc $\geq$ ngày bắt đầu", "mỗi phòng có tối đa 50 nhân viên"). Mô hình quan hệ không tự suy ra được chúng — người thiết kế phải khai báo bằng `CHECK`, `ASSERTION` hoặc trigger.

Ràng buộc toàn vẹn là lớp phòng vệ *cuối cùng* nằm ngay trong CSDL: nó vẫn có hiệu lực kể cả khi ứng dụng bị chiếm quyền hoặc bị tiêm SQL.

Xử lý khi vi phạm toàn vẹn tham chiếu

Khi xóa hoặc sửa bộ *được tham chiếu*, hệ quản trị CSDL cần biết phải làm gì:

Phương án	Hành vi
`NO ACTION / RESTRICT`	Từ chối thao tác (mặc định của chuẩn SQL)
`CASCADE`	Xóa/sửa lan sang các bộ đang tham chiếu
`SET NULL`	Đặt khóa ngoại về `NULL` (chỉ được nếu cột cho phép `NULL`)
`SET DEFAULT`	Đặt về giá trị mặc định

```
CREATE TABLE THAMGIA (
  MaNV  CHAR(8),
  MaDA  CHAR(8),
  SoGio DECIMAL(4,1) CHECK (SoGio >= 0),
  PRIMARY KEY (MaNV, MaDA),
  FOREIGN KEY (MaNV) REFERENCES NHANVIEN(MaNV) ON DELETE CASCADE,
  FOREIGN KEY (MaDA) REFERENCES DUAN(MaDA)     ON DELETE RESTRICT
);
```

`CASCADE` rất tiện nhưng nguy hiểm: một lệnh `DELETE` hợp lệ có thể xóa dây chuyền hàng loạt bản ghi ở các bảng khác. Với dữ liệu cần lưu vết, hãy dùng `RESTRICT` kèm cột đánh dấu "đã hủy" thay vì xóa thật.

Thiết kế mức logic

Mục tiêu

Chuyển thiết kế *mức khái niệm* (lược đồ ER) thành tập *lược đồ quan hệ* kèm đầy đủ khóa và ràng buộc, sẵn sàng cài đặt trên một HQT CSDL cụ thể.

- **Đầu vào:** lược đồ ER.
- **Đầu ra:** lược đồ quan hệ + ràng buộc.

Vai trò của công cụ CASE

Công cụ **CASE** (*Computer-Aided Software Engineering*) giúp:

- Tự động hóa phần lớn việc ánh xạ.
- Kiểm tra tính nhất quán của lược đồ.
- Sinh câu lệnh DDL và đồng bộ với máy chủ CSDL.

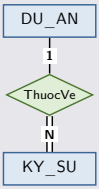
Nguyên tắc chung

Việc ánh xạ tuân theo một **tập quy tắc xác định** nhằm bảo toàn ngữ nghĩa: mọi thông tin biểu diễn được trên lược đồ ER phải biểu diễn được trên lược đồ quan hệ, và mọi ràng buộc ER phải trở thành một ràng buộc quan hệ tương ứng.

Ví dụ mở đầu

Bài toán: quản lý các **dự án AI** và các **kỹ sư** tham gia.

1. Mức khái niệm (ER)



- DU_AN: MaDA, TenDA.
- KY_SU: MaKS, HoTen, ChuyenMon.

2. Mức logic (lược đồ quan hệ)

DU_AN(MaDA, TenDA)
KY_SU(MaKS, HoTen, ChuyenMon, *MaDA*)

MaKS	HoTen	ChuyenMon	*MaDA*
KS01	An	CV	AI_01
KS02	Bình	NLP	AI_01

Hai quy tắc đã dùng

(1) Mỗi kiểu thực thể → một quan hệ. **(2)** Liên kết 1:*N* → đưa khóa chính của phía **1** vào quan hệ phía **N** làm khóa ngoại. Vì KY_SU tham gia toàn bộ, *MaDA* khai báo NOT NULL.

Vì sao không phó mặc cho công cụ CASE?

1. **Cấu trúc phức tạp:** công cụ tự động thường xử lý kém các tình huống giàu ngữ nghĩa — liên kết *tam phân*, *lớp cha/lớp con* (Specialization/Generalization), và các ràng buộc nghiệp vụ đặc thù.
2. **Nhiều phương án đều hợp lệ:** một lược đồ ER có thể ánh xạ theo vài cách khác nhau mà cách nào cũng đúng về hình thức. Người thiết kế phải chọn dựa trên *mẫu truy vấn thực tế*, *hiệu năng* và *khả năng mở rộng*.
3. **Kiểm soát chất lượng:** kết quả của công cụ chỉ là bản nháp; vẫn phải kiểm chứng về chuẩn hóa và toàn vẹn dữ liệu trước khi triển khai.

Ví dụ cụ thể cho ý (2)

Một liên kết 1:1 có thể ánh xạ thành: (a) khóa ngoại đặt ở quan hệ A; (b) khóa ngoại đặt ở quan hệ B; hoặc (c) *gộp* cả hai thành một quan hệ duy nhất. Cả ba đều bảo toàn thông tin, nhưng số `NULL` và chi phí phép nối thì khác hẳn nhau.

"Công cụ là trợ thủ; tư duy hệ thống của kỹ sư mới là quyết định."

Thuật toán ánh xạ - tổng quan 7 bước

Bước	Đối tượng trong ER	Kết quả trong mô hình quan hệ
1	Kiểu thực thể mạnh	Một quan hệ; khóa thực thể $\rightarrow$ khóa chính
2	Kiểu thực thể yếu	Một quan hệ; PK = PK thực thể chủ + khóa bộ phận
3	Liên kết nhị phân	1:N, 1:1 $\rightarrow$ khóa ngoại; M:N $\rightarrow$ quan hệ mới
4	Thực thể kết hợp	Một quan hệ nối chứa các khóa ngoại
5	Liên kết đệ quy	Khóa ngoại đệ quy, hoặc quan hệ nối
6	Liên kết nhiều ngôi	Một quan hệ chứa khóa ngoại của mọi phía
7	Lớp cha / lớp con	Nhiều phương án — xem bước 7

Ba nguyên tắc xuyên suốt

1. Thuộc tính **phức hợp** $\rightarrow$ chỉ đưa các *thành phần đơn* vào quan hệ.
2. Thuộc tính **đa trị** $\rightarrow$ luôn tạo một *quan hệ riêng*.
3. Thuộc tính **dẫn xuất** $\rightarrow$ thường *không* lưu; tính lại khi cần hoặc dùng khung nhìn.

Phân loại kiểu thực thể khi ánh xạ

Nguyên tắc: cách ánh xạ một kiểu thực thể phụ thuộc vào *bản chất tồn tại* của nó trong lược đồ ER.

1. **Kiểu thực thể mạnh**
   - Tồn tại độc lập, tự định danh bằng khóa riêng. Ký hiệu: hình chữ nhật nét đơn.
   - **Ánh xạ:** một quan hệ riêng với các thuộc tính tương ứng.
2. **Kiểu thực thể yếu**
   - Không có khóa riêng; tồn tại phụ thuộc vào một *thực thể chủ* (owner). Ký hiệu: hình chữ nhật nét đôi.
   - **Ánh xạ:** khóa chính = khóa chính của thực thể chủ + khóa bộ phận.
3. **Thực thể kết hợp** (*associative entity*)
   - Xuất hiện khi một liên kết $M{:}N$ có thuộc tính riêng, hoặc khi bản thân liên kết cần được tham chiếu bởi liên kết khác.
   - **Ánh xạ:** một "bảng nối" (junction table) chứa khóa ngoại của các phía tham gia.

[Lưu ý] Ký hiệu *hình thoi lồng trong hình chữ nhật* cho thực thể kết hợp thuộc ký pháp của giáo trình Hoffer, không có trong ký pháp Chen đã dùng ở bài trước. Về bản chất nó chính là một liên kết $M{:}N$ được "nâng cấp" thành kiểu thực thể.

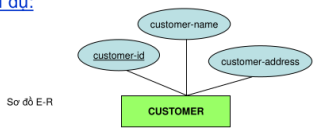
Bước 1: Ánh xạ kiểu thực thể mạnh

- Mỗi kiểu thực thể mạnh → **một lược đồ quan hệ**; tên quan hệ thường lấy theo tên kiểu thực thể.
- Mỗi thuộc tính **đơn** của kiểu thực thể → một thuộc tính của quan hệ.
- Thuộc tính **khóa** (gạch chân trên ERD) → **khóa chính**; các khóa dự bị còn lại khai báo `UNIQUE`.
- Thuộc tính **dẫn xuất** thường *không* được đưa vào — nếu có đưa vào (vì lý do hiệu năng) thì phải có cơ chế cập nhật để giữ nhất quán.

Ví dụ

`CUSTOMER(customer-id, customer-name, customer-address)`

Ví dụ:



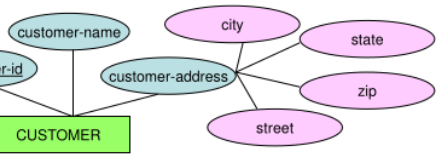
Sơ đồ E-R

Quan hệ CUSTOMER

Bước 1 (tiếp): thuộc tính phức hợp

- Với thuộc tính **phức hợp**, *chỉ các thành phần đơn* được đưa vào quan hệ; bản thân tên thuộc tính phức hợp *biến mất* khỏi lược đồ quan hệ.
- Ví dụ: `customer-address` gồm `street`, `city`, `state`, `zip` ⇒ quan hệ có bốn cột, không có cột `customer-address`.

Ví dụ:



Sơ đồ E-R

CUSTOMER

customer-id	customer-name	street	city	state	zip

Quan hệ CUSTOMER

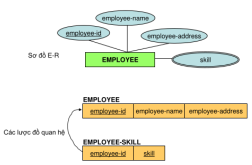
Bước 1 (tiếp): thuộc tính đa trị

- Một thuộc tính **đa trị** luôn sinh ra **hai** lược đồ quan hệ:
  - Quan hệ thứ nhất: chứa mọi thuộc tính của kiểu thực thể, *trừ* thuộc tính đa trị.
  - Quan hệ thứ hai: có đúng hai thành phần tạo thành khóa chính — (a) khóa chính của quan hệ thứ nhất, *đồng thời là khóa ngoại*; (b) chính thuộc tính đa trị.
- Tên quan hệ thứ hai nên phản ánh ngữ nghĩa của thuộc tính đa trị (`EMPLOYEE-SKILL`, không phải `BANG2`).

Vì sao khóa chính gồm *cả hai* thuộc tính?

Một nhân viên có nhiều kỹ năng (nên `employee-id` một mình không đủ), và nhiều nhân viên cùng có một kỹ năng (nên `skill` một mình cũng không đủ).

Ví dụ cho trường hợp thuộc tính đa trị:



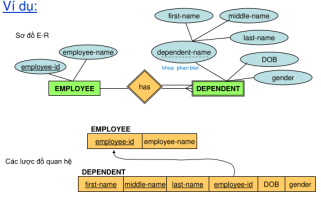
Bước 2: Ánh xạ kiểu thực thể yếu

- Giả sử quan hệ của **thực thể chủ** đã được tạo ở bước 1.
- Tạo một quan hệ mới cho kiểu thực thể yếu, đưa vào mọi thuộc tính đơn (hoặc các thành phần đơn của thuộc tính phức hợp).
- Thêm khóa chính của quan hệ chủ vào làm **khóa ngoại**.
- **Khóa chính** của quan hệ mới = khóa ngoại đó + *khóa bộ phận* của thực thể yếu.

Hai ràng buộc bắt buộc đi kèm

1. Khóa ngoại `NOT NULL` — vì nó nằm trong khóa chính (toàn vẹn thực thể), và vì thực thể yếu luôn tham gia toàn bộ vào liên kết định danh.
2. Nên khai báo `ON DELETE CASCADE`: xóa thực thể chủ thì thực thể yếu phụ thuộc nó không còn ý nghĩa tồn tại.

Ví dụ:



Bước 3: Liên kết nhị phân 1:*N*

- Tạo lược đồ quan hệ cho mỗi kiểu thực thể tham gia (theo bước 1).
- Đưa **khóa chính của phía "1"** vào quan hệ ở **phía "N"** để làm **khóa ngoại**.
- Thuộc tính của *liên kết* (nếu có) cũng được đưa vào quan hệ phía "N", dưới dạng thuộc tính thường.
- Không tạo quan hệ mới — đây là điểm khác cơ bản so với *M*:*N*.

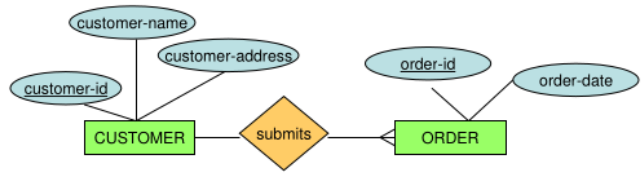
Ràng buộc tham gia quyết định `NULL`

Nếu phía "N" tham gia **toàn bộ** (mỗi nhân viên *phải* thuộc một phòng) $\Rightarrow$ khóa ngoại khai báo `NOT NULL`. Nếu tham gia **bộ phận** $\Rightarrow$ cho phép `NULL`. Bỏ qua chi tiết này là mất thông tin so với lược đồ ER.

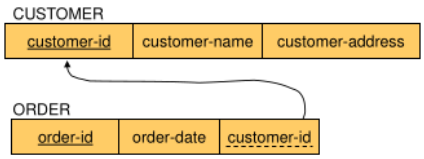
Bước 3: ví dụ 1:*N*

Ví dụ cho trường hợp quan hệ 1-N hai ngôi:

Sơ đồ E-R



Các lược đồ quan hệ



Đọc hình: `customer-id` trong `ORDER` được gạch chân nét đứt = khóa ngoại. Một đơn hàng luôn thuộc về một khách hàng $\Rightarrow$ cột này `NOT NULL`.

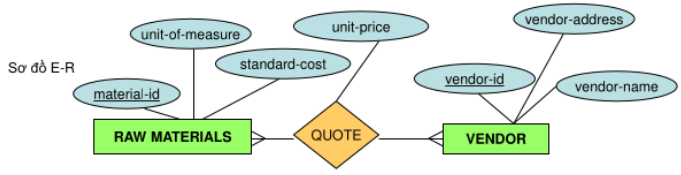
Bước 3: Liên kết nhị phân $M{:}N$

- Cho liên kết $M{:}N$ giữa hai kiểu thực thể A và B.
- **Bắt buộc** tạo thêm một quan hệ mới C ("bảng nối", *junction table*).
- **Khóa chính** của C = hợp khóa chính của A và của B; đồng thời mỗi phần là một **khóa ngoại** trỏ về quan hệ tương ứng.
- Mọi thuộc tính *của liên kết* được đưa vào C dưới dạng thuộc tính thường.

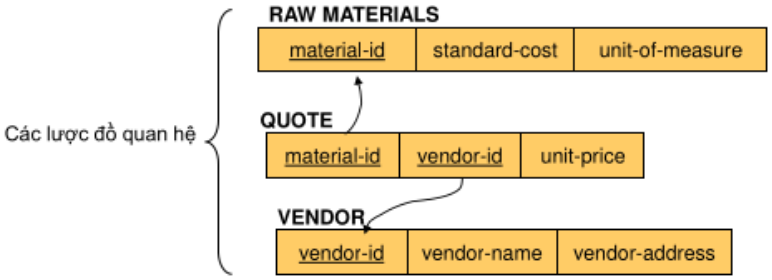
Vì sao không thể dùng khóa ngoại như trường hợp $1{:}N$?

Vì một ô chỉ chứa được *một* giá trị nguyên tố. Nếu đặt `vendor-id` vào `RAW_MATERIALS` thì mỗi vật tư chỉ báo giá được cho *một* nhà cung cấp — đúng nghĩa $1{:}N$, không còn là $M{:}N$ nữa.

Ví dụ cho trường hợp liên kết N-N hai ngôi:



Bước 3: ví dụ *M*:*N*



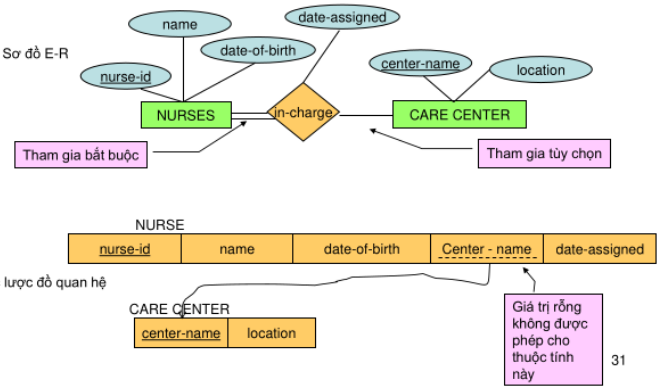
Kết quả: `QUOTE(material-id, vendor-id, unit-price)` — hai cột vừa hợp thành khóa chính, vừa là hai khóa ngoại riêng biệt.

Bước 3: Liên kết nhị phân 1:1

- Tạo quan hệ cho hai kiểu thực thể tham gia, rồi đặt khóa chính của *một* quan hệ sang quan hệ kia làm khóa ngoại (kèm ràng buộc `UNIQUE`).
- **Chọn phía nào?** Trong liên kết 1:1 thường một phía tham gia *toàn bộ*, phía kia *bộ phận*. Hãy đặt khóa ngoại vào quan hệ của phía **tham gia toàn bộ** — như vậy cột khóa ngoại không bao giờ `NULL`.
- Thuộc tính của liên kết được đưa vào *cùng* quan hệ đó, dưới dạng **thuộc tính thường**.
- Nếu *cả hai* phía đều tham gia toàn bộ, có thể **gộp** hai kiểu thực thể thành một quan hệ duy nhất.

Bước 3: ví dụ 1:1

Ví dụ cho trường hợp liên kết 1-1 hai ngôi:



Đọc hình: `NURSES` tham gia bắt buộc nên khóa ngoại `Center-name` được đặt vào `NURSE` và *không* cho phép `NULL`; `date-assigned` là thuộc tính của liên kết, đi kèm khóa ngoại.

Bước 4: Ánh xạ thực thể kết hợp

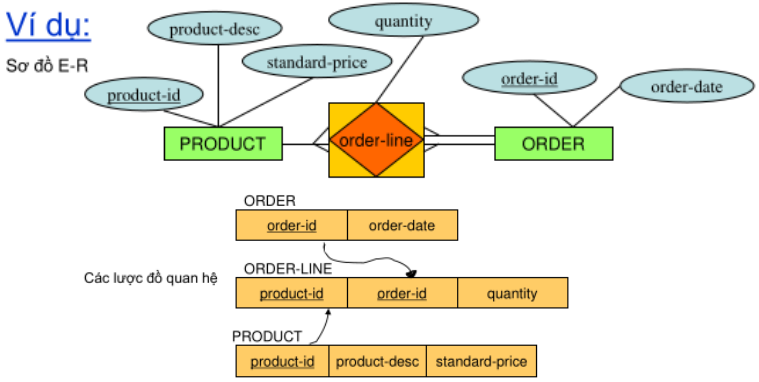
- Cách làm tương tự liên kết $M{:}N$, gồm hai bước:
  1. Tạo **ba** quan hệ: hai quan hệ cho các kiểu thực thể tham gia, và một quan hệ thứ ba cho thực thể kết hợp — gọi là **quan hệ kết hợp**.
  2. Cách chọn khóa chính cho quan hệ kết hợp phụ thuộc vào việc lược đồ ER *có gán định danh riêng* cho thực thể kết hợp hay không:
     (a) **Không gán định danh** — dùng khóa tổ hợp.
     (b) **Có gán định danh** — dùng khóa thay thế (surrogate key).

Khi nào nên nâng liên kết thành thực thể kết hợp?

Khi liên kết có *nhiều* thuộc tính riêng, khi nó cần *được tham chiếu* bởi một liên kết khác, hoặc khi cùng một cặp thực thể có thể liên kết *nhiều lần* (mà mô hình ER chuẩn không cho phép).

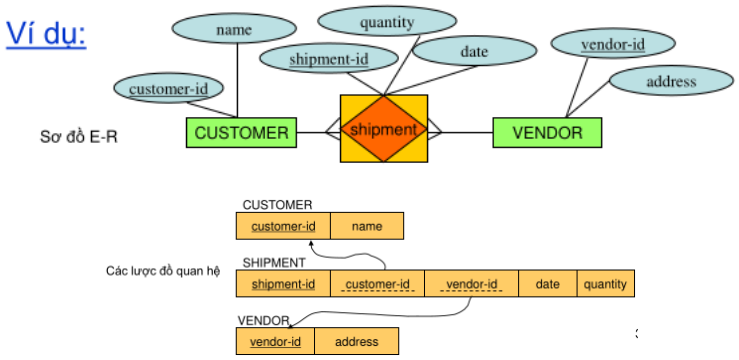
Bước 4 (a): không gán định danh

Khóa chính ngầm định của quan hệ kết hợp gồm các thuộc tính khóa chính lấy từ hai quan hệ còn lại; đồng thời chúng là khóa ngoại tham chiếu tới hai quan hệ đó.



Bước 4 (b): có gán định danh

Tạo quan hệ kết hợp với **khóa chính là định danh được gán** trên lược đồ ER (ví dụ `shipment-id`); khóa chính của hai kiểu thực thể tham gia trở thành **khóa ngoại**.



[Lưu ý] Khi chuyển sang khóa thay thế, ta **đánh mất** ràng buộc duy nhất của cặp (`customer-id`, `vendor-id`). Nếu nghiệp vụ vẫn yêu cầu mỗi cặp chỉ xuất hiện một lần, phải bổ sung `UNIQUE(customer-id, vendor-id)`.

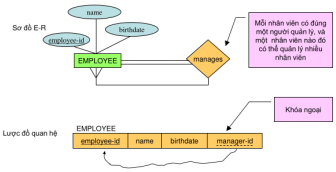
Bước 5: Liên kết đệ quy loại 1:*N*

- Ánh xạ kiểu thực thể thành quan hệ theo bước 1.
- Thêm vào *chính quan hệ đó* một thuộc tính khóa ngoại tham chiếu tới khóa chính của nó — gọi là **khóa ngoại đệ quy** (*recursive foreign key*).
- Khóa ngoại phải có **cùng miền giá trị** với khóa chính, nhưng *nên đặt tên khác* để thể hiện vai trò (`manager-id`, không phải `employee-id`).

Khóa ngoại đệ quy gần như luôn phải cho phép `NULL`

Nhân viên ở đỉnh sơ đồ tổ chức không có người quản lý. Nếu khai báo `NOT NULL`, ta sẽ *không thể chèn được bộ đầu tiên* — một lỗi rất hay gặp khi sinh viên cài đặt.

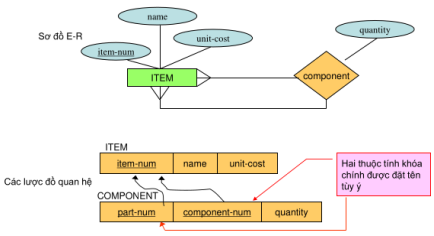
Ví dụ trường hợp ánh xạ quan hệ đệ quy loại 1-N



Bước 5: Liên kết đệ quy loại *M*:*N*

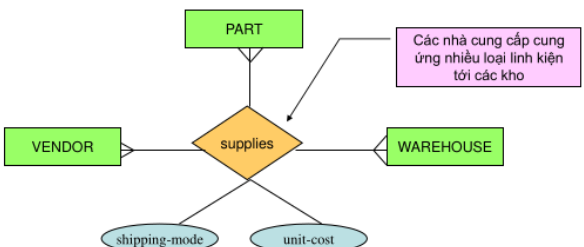
- Tạo **hai** quan hệ: một cho kiểu thực thể, một cho liên kết.
- Khóa chính của quan hệ liên kết gồm **hai thuộc tính**, cả hai đều lấy giá trị từ khóa chính của quan hệ thực thể — nhưng *phải đặt tên khác nhau* để phân biệt hai vai trò (`part-num` và `component-num`).
- Mọi thuộc tính *của liên kết* (ví dụ `quantity`) được đưa vào quan hệ liên kết.

Ví dụ trường hợp ánh xạ quan hệ đệ quy loại N-N

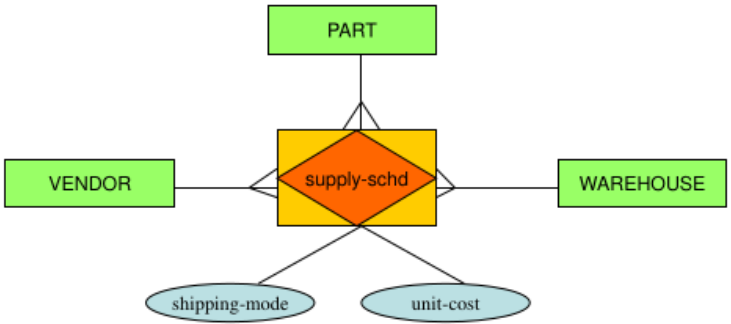


Bước 6: Liên kết nhiều ngôi

- Liên kết **tam phân** là liên kết giữa ba kiểu thực thể; tổng quát là liên kết bậc *n*.
- Cách làm: chuyển liên kết nhiều ngôi thành một **thực thể kết hợp**, rồi áp dụng bước 4.
- Tạo một quan hệ mới; khóa chính ngầm định gồm khóa chính của *mọi* kiểu thực thể tham gia, mỗi phần đồng thời là một khóa ngoại. Mọi thuộc tính của liên kết trở thành thuộc tính thường.



Bước 6: chuyển thành thực thể kết hợp



Chú ý: `shipping-mode` và `unit-cost` phụ thuộc vào *cả ba* phía — đó chính là dấu hiệu cho thấy đây là liên kết tam phân thật sự, chứ không phải ba liên kết nhị phân rời rạc.

Bước 6: khóa chính của quan hệ nhiều ngôi

- Trường hợp tổng quát: khóa chính $= K_1 \cup K_2 \cup \cdots \cup K_n$.
- **Nhưng** nếu một phía tham gia với lực lượng **1**, khóa của phía đó *bị loại* khỏi khóa chính (lập luận giống hệt trường hợp 1:N nhị phân).
- Ngược lại, nếu cùng một bộ ba thực thể có thể liên kết *nhiều lần*, thì hợp các khóa *không đủ* — phải bổ sung thuộc tính phân biệt vào khóa, hoặc dùng khóa thay thế.

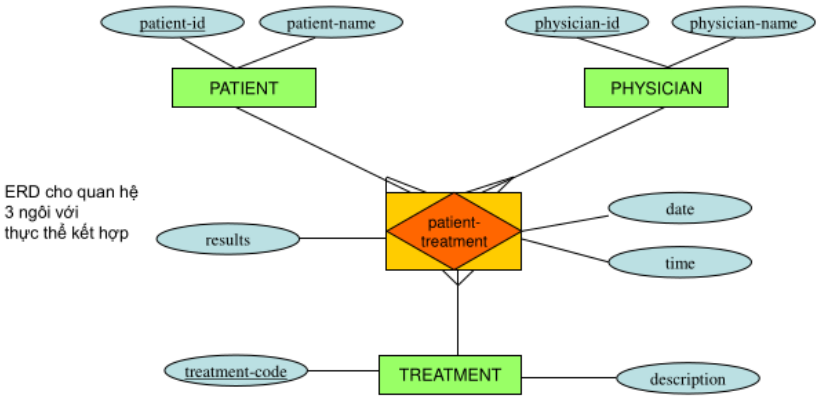
Kiểm tra lại ví dụ `PATIENT-TREATMENT` ở slide sau

Lược đồ cho khóa chính là `(patient-id, physician-id, treatment-code)`, kéo theo ngữ nghĩa: *một bệnh nhân chỉ được một bác sĩ thực hiện một loại điều trị đúng một lần trong đời*. Nếu nghiệp vụ không như vậy, phải đưa `date` (và có thể cả `time`) **vào khóa chính**.

[Lưu ý] Đây đúng là tình huống `ENROLLS` đã bàn ở bài ER: mô hình không cho phép hai liên kết trùng nhau giữa cùng một bộ thực thể. Hãy luôn tự hỏi: *"cặp/bộ này có thể lặp lại không?"* trước khi chốt khóa chính.

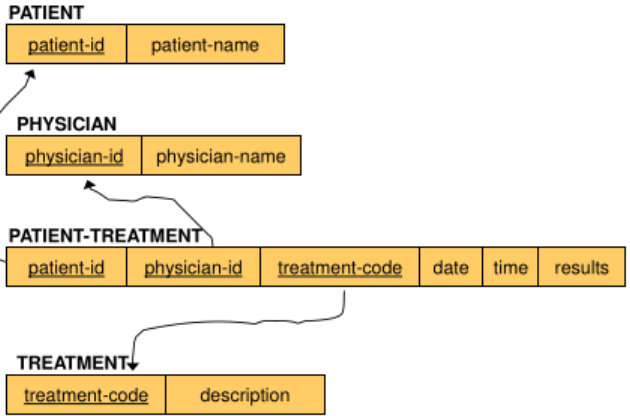
Bước 6: ví dụ liên kết tam phân

- Ví dụ, cho quan hệ 3 ngôi:



Bước 6: kết quả ánh xạ

Các lược đồ quan hệ



Đọc hình: ba cột gạch chân của `PATIENT-TREATMENT` vừa hợp thành khóa chính, vừa là ba khóa ngoại độc lập nhau.

Bước 7: Ánh xạ liên kết lớp cha / lớp con

Phương án mặc định (dùng được cho mọi trường hợp):

1. Tạo một quan hệ riêng cho lớp cha và cho *từng* lớp con.
2. Đưa các thuộc tính *chung* của mọi lớp con (kể cả khóa chính) vào quan hệ lớp cha.
3. Khóa chính của lớp cha trở thành **khóa chính** của mỗi quan hệ lớp con, *đồng thời* là **khóa ngoại** trỏ về lớp cha.
4. Thêm vào lớp cha một (hoặc nhiều) thuộc tính đóng vai trò **thành phần phân biệt** (*discriminator*) để biết mỗi bộ thuộc lớp con nào.

Điểm mấu chốt ở bước 3

Cột `h-employee-id` trong `HOURLY-EMP` *vừa* là khóa chính *vừa* là khóa ngoại. Chính ràng buộc kép này bảo đảm: mỗi nhân viên theo giờ phải tồn tại trong bảng `EMPLOYEE`, và không thể xuất hiện hai lần.

Bước 7: bốn phương án và điều kiện áp dụng

PA	Cách làm	Điều kiện / đánh đổi
A	Quan hệ cho lớp cha + mỗi lớp con	Dùng được *mọi* trường hợp; nhiều phép nối
B	Chỉ các lớp con (mỗi lớp con chứa cả thuộc tính cha)	Chỉ khi phân hoạch **toàn bộ** và **rời nhau**
C	Một quan hệ duy nhất + một thuộc tính phân biệt	Chỉ khi các lớp con **rời nhau**; sinh nhiều `NULL`
D	Một quan hệ duy nhất + nhiều cột luận lý	Dùng được khi các lớp con **giao nhau**

Cách chọn

- Lớp con có *nhiều* thuộc tính riêng, truy vấn thường chỉ chạm một lớp con $\Rightarrow$ chọn **A** hoặc **B**.
- Lớp con có *ít* thuộc tính riêng, truy vấn hay gộp cả lớp cha $\Rightarrow$ chọn **C** (tránh phép nối).
- Lớp con *giao nhau* (một linh kiện vừa tự sản xuất vừa mua ngoài) $\Rightarrow$ **B** và **C** đều sai; chọn **A** hoặc **D**.

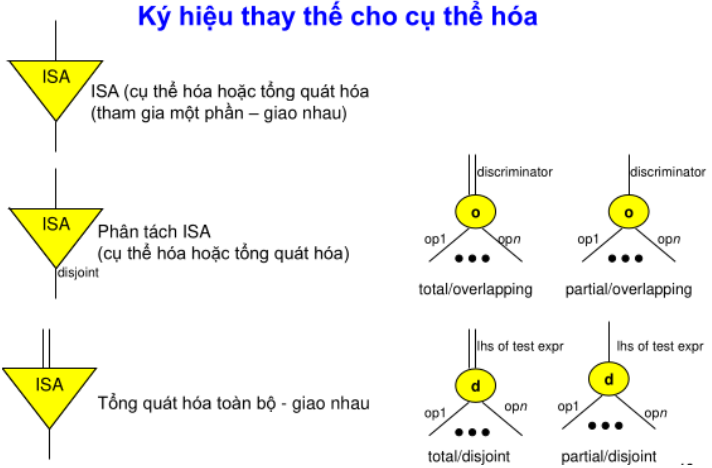
Bước 7: thành phần phân biệt lớp con

- Khi thêm một thể hiện của lớp cha vào CSDL, làm sao biết nó thuộc lớp con nào?
- Cách phổ biến là dùng một **thành phần phân biệt**: một thuộc tính của lớp cha mà giá trị của nó xác định lớp con tương ứng (dùng khi việc xác định lớp con dựa trên điều kiện về thuộc tính).
- Hai trường hợp: các lớp con **rời nhau** (*disjoint*) và các lớp con **giao nhau** (*overlapping*).

Hệ quả về cách cài đặt

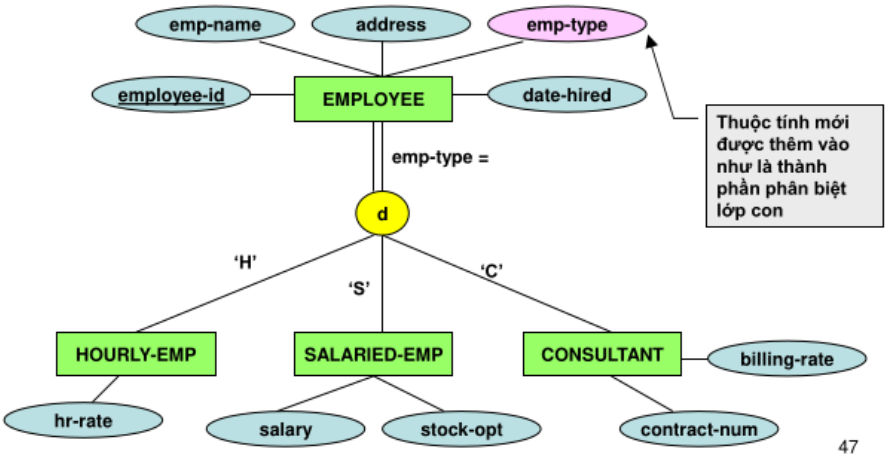
Rời nhau → *một* cột phân biệt, ví dụ `emp-type CHAR(1) CHECK (emp-type IN ('H','S','C'))`.
Giao nhau → *nhiều* cột luận lý độc lập, ví dụ `is-manufactured BOOLEAN`, `is-purchased BOOLEAN` — vì một bộ có thể đúng với nhiều lớp con cùng lúc.

Bước 7: ký hiệu cho cụ thể hóa / tổng quát hóa

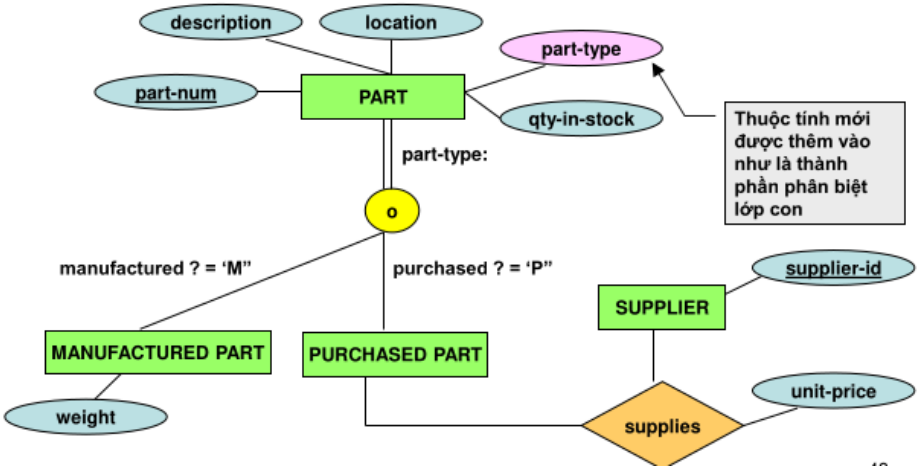


Đọc ký hiệu: vòng tròn d = rời nhau (*disjoint*), o = giao nhau (*overlapping*); nét **đôi** ở nhánh nối lên lớp cha = cụ thể hóa **toàn bộ** (mọi thể hiện lớp cha phải thuộc ít nhất một lớp con), nét đơn = **bộ phận**.

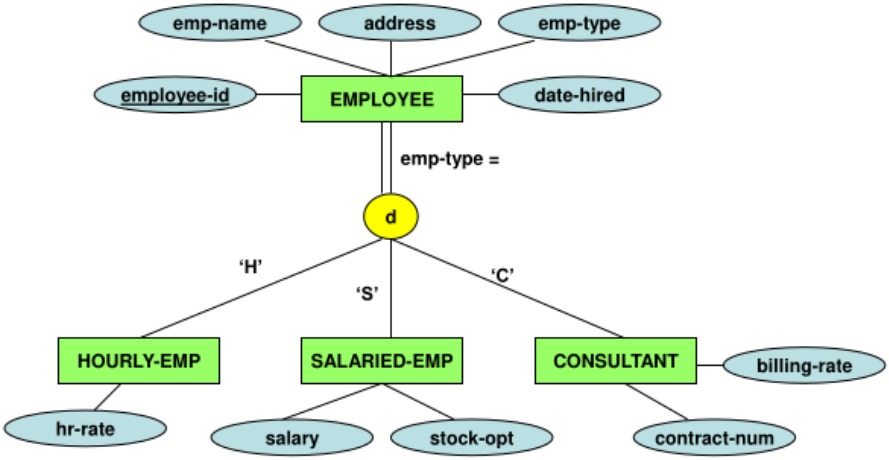
Bước 7: ví dụ lớp con rời nhau



Bước 7: ví dụ lớp con giao nhau

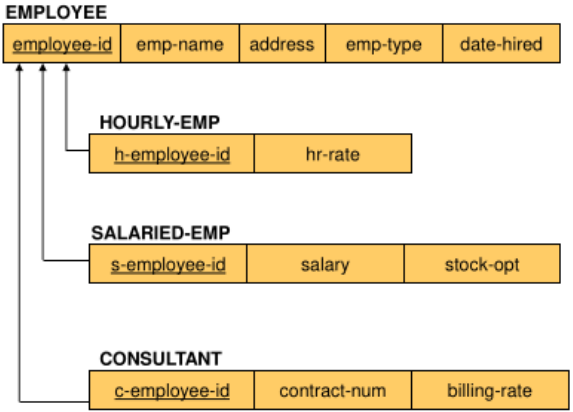


Bước 7: lược đồ ER cần ánh xạ



Bước 7: kết quả

Lược đồ quan hệ kết quả



Chú ý: `h-employee-id`, `s-employee-id`, `c-employee-id` vừa là khóa chính của bảng mình, vừa là khóa ngoại trỏ về `EMPLOYEE` — đây là phương án **A** ở bảng so sánh phía trước.

Áp dụng: ánh xạ lược đồ ER bài toán công ty

Lấy lại lược đồ ER đã xây dựng ở bài trước, áp dụng đủ 7 bước:

Quan hệ	**Nguồn gốc**
NHANVIEN(<u>MaNV</u>, Ho, TenDem, Ten, DiaChi, Luong, GioiTinh, NgaySinh, *MaPB*, *MaNGS*)	B1 (HoTen phức hợp → tách cột); B3 (1:*N* LAMVIEC); B5 (đệ quy GIAMSAT)
SOTHICH_NV(<u>MaNV</u>, <u>SoThich</u>)	B1 — thuộc tính đa trị
PHONGBAN(<u>MaPB</u>, TenPB, *MaTruongPhong*, NgayNhamChuc)	B1; B3 (1:1 QUANLY + thuộc tính liên kết)
DIADIEM_PB(<u>MaPB</u>, <u>DiaDiem</u>)	B1 — thuộc tính đa trị
DUAN(<u>MaDA</u>, TenDA, DiaDiem, *MaPB*)	B1; B3 (1:*N* PHUTRACH)
THAMGIA(<u>MaNV</u>, <u>MaDA</u>, SoGio)	B3 — liên kết *M*:*N*
THANNHAN(<u>MaNV</u>, <u>Ten</u>, GioiTinh, NgaySinh, QuanHe)	B2 — thực thể yếu

Gạch chân = khóa chính; *nghiêng* = khóa ngoại. Thuộc tính dẫn xuất (Tuoi, SoNhanVien) không được lưu.

Áp dụng: các ràng buộc đi kèm

Khóa ngoại	**Tham chiếu**	NULL?
NHANVIEN.MaPB	PHONGBAN	NOT NULL (tham gia toàn bộ)
NHANVIEN.MaNGS	NHANVIEN	cho phép NULL (đỉnh sơ đồ tổ chức)
PHONGBAN.MaTruongPhong	NHANVIEN	NOT NULL (mỗi phòng phải có trưởng phòng)
DUAN.MaPB	PHONGBAN	NOT NULL
THAMGIA.MaNV / MaDA	tương ứng	NOT NULL (nằm trong khóa chính)
THANNHAN.MaNV	NHANVIEN	NOT NULL, ON DELETE CASCADE

Bài toán "con gà — quả trứng"

NHANVIEN.MaPB NOT NULL trỏ sang PHONGBAN, còn PHONGBAN.MaTruongPhong NOT NULL lại trỏ ngược về NHANVIEN $\Rightarrow$ *không chèn được bộ nào đầu tiên*. Cách xử lý: khai báo ràng buộc DEFERRABLE INITIALLY DEFERRED rồi chèn cả hai trong một giao dịch, hoặc tạm thời cho phép một trong hai cột nhận NULL.

Áp dụng: sinh câu lệnh DDL

```
CREATE TABLE PHONGBAN (
  MaPB          CHAR(4)      PRIMARY KEY,
  TenPB         NVARCHAR(50) NOT NULL UNIQUE,
  MaTruongPhong CHAR(8)      NOT NULL,
  NgayNhamChuc  DATE
);

CREATE TABLE NHANVIEN (
  MaNV     CHAR(8)      PRIMARY KEY,
  Ho       NVARCHAR(20), TenDem NVARCHAR(20), Ten NVARCHAR(20) NOT NULL,
  Luong    DECIMAL(12,0) CHECK (Luong >= 0),
  MaPB     CHAR(4)      NOT NULL REFERENCES PHONGBAN(MaPB),
  MaNGS    CHAR(8)      REFERENCES NHANVIEN(MaNV)   -- de quy, cho phep NULL
);

ALTER TABLE PHONGBAN
  ADD FOREIGN KEY (MaTruongPhong) REFERENCES NHANVIEN(MaNV);
```

Chú ý thứ tự: ràng buộc vòng giữa hai bảng được tách ra thành `ALTER TABLE` sau khi cả hai bảng đã tồn tại.

Tổng kết

1. Quan hệ là **tập con của tích Descartes** các miền giá trị — mọi đặc tính của quan hệ đều suy ra từ đây.
2. Phân biệt **lược đồ** (cố định) với **thể hiện** (thay đổi); ràng buộc phát biểu trên lược đồ và phải đúng với mọi thể hiện.
3. Ba ràng buộc toàn vẹn: **miền giá trị**, **toàn vẹn thực thể** (khóa chính khác `NULL`), **toàn vẹn tham chiếu** (khóa ngoại).
4. Ánh xạ ER → quan hệ theo **7 bước**; điểm quyết định luôn là *tỉ lệ lực lượng* và *ràng buộc tham gia*.
5. 1:N và 1:1 → khóa ngoại; M:N và nhiều ngôi → quan hệ mới. Thuộc tính đa trị → luôn là quan hệ riêng.
6. Ràng buộc tham gia **toàn bộ** → khóa ngoại `NOT NULL`. Bỏ sót chi tiết này là đánh mất thông tin của lược đồ ER.
7. Bảng SQL không phải là quan hệ toán học — nó cho phép bộ trùng và có `NULL`.

Bài tập

Bài tập trên lớp

1. Ánh xạ lược đồ ER `CARD ACCOUNT` (slide "Bước 7: lược đồ ER cần ánh xạ") sang lược đồ quan hệ, ghi rõ khóa chính, khóa ngoại và cột nào cho phép `NULL`. Giải thích vì sao chọn phương án A/B/C/D.
2. Cho liên kết `DANGKY` giữa `SINHVIEN` và `HOCPHAN`, trong đó một sinh viên *có thể học lại* cùng một học phần ở kỳ khác. Đề xuất lược đồ quan hệ và khóa chính đúng.
3. Viết câu lệnh `CREATE TABLE` cho quan hệ `THANNHAN` và `THAMGIA` của bài toán công ty, đủ ràng buộc toàn vẹn.

Câu hỏi ôn tập

1. Vì sao thứ tự các bộ trong một quan hệ là vô nghĩa, trong khi kết quả `SELECT` lại có thứ tự?
2. Một liên kết 1:1 toàn bộ ở cả hai phía nên ánh xạ thế nào? Nêu ưu và nhược điểm.
3. Vì sao khóa ngoại đệ quy hầu như luôn phải cho phép `NULL`?
4. Phân biệt `ON DELETE CASCADE` và `ON DELETE SET NULL`. Với bảng `THANNHAN` thì chọn cái nào, vì sao?

Tài liệu tham khảo

- E. F. Codd. *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, 13(6), 1970.
- R. Elmasri, S. B. Navathe. *Fundamentals of Database Systems*, 7th ed., Pearson, 2016 (Ch. 5, 9).
- T. Connolly, C. Begg. *Database Systems: A Practical Approach*, 6th ed., Pearson, 2015 (Ch. 4, 16).
- A. Silberschatz, H. F. Korth, S. Sudarshan. *Database System Concepts*, 7th ed., McGraw-Hill, 2020 (Ch. 2, 6).
- C. J. Date. *An Introduction to Database Systems*, 8th ed., Addison-Wesley, 2003.